iDEFT: Intelligent Backup-Node Selection for Fault-Tolerant Network Function State Replication

***Abstract*—Network Function Virtualization (NFV) systems that provide fault tolerance through state replication (OpenNF, Pico Replication, FTMB, REINFORCE, and DEFT among them) all replicate a primary network function's (NF) state to a backup instance. None of them specify how that backup instance is chosen. In DEFT, for example, the Failure Detection Unit simply assigns a new secondary NF with no stated criterion. This paper closes that gap with iDEFT (intelligent DEFT). We design three backup-selection strategies. Each computes the primary-to-backup mapping DEFT already consumes, so it replaces the static, hand-assigned mapping with no other change to the system. The first is a globally optimal assignment under per-backup capacity constraints, solved exactly via the Hungarian algorithm. Because it requires centralized computation, we use it only as an upper-bound benchmark, not a deployment candidate. The second is a multi-criteria greedy heuristic scoring health, remaining capacity, network latency, and hop-count. The third is a Proximal Policy Optimization (PPO) reinforcement-learning policy trained on the same objective with invalid-action masking. We evaluate all three against DEFT's static mapping, naive load-spreading baselines, and decentralized hash-based placement (rendezvous and consistent hashing). All methods drive DEFT's real, unmodified two-phase-commit (2PC) backup implementation under controlled concurrent load. We test across four topology scales (9, 16, 32, and 128 hosts) with rigorous multi-seed statistical testing (Mann-Whitney U tests, rank-biserial effect sizes, bootstrap confidence intervals). We find that feature-aware methods reliably avoid degraded backup hosts and substantially reduce tail latency compared to blind load-balancing, once hosts differ in quality. A well-designed deterministic heuristic method tracks the Hungarian-optimal bound closely at every scale tested. We run a systematic sensitivity analysis over the scoring weights and load-decay mechanics of this multi-criteria heuristic method. Its degraded-host-avoidance guarantee is robust across the entire weight space. Its load-balancing quality is driven almost entirely by one term, remaining capacity, so the published default is a principled, non-fragile choice rather than an arbitrary one. A learned RL policy (PPO) matches the feature-aware methods up to 32 hosts; at 128 hosts it avoids degraded hosts completely but load-balances less tightly than the heuristic. A cross-algorithm check with DQN shows the same concentration behavior at scale, and more sharply, under the same widened-exploration remedy, indicating the gap follows from the problem formulation rather than from one algorithm. We leave a fuller study of RL for backup-node selection in this setting, including reward shaping that directly penalizes load concentration, to future work. The code, trained models, and the saved output of every experiment are available for reproduction at https://bijoy111.github.io/ideft/.**

***Index Terms*—network function virtualization, fault tolerance, state replication, backup node selection, replica placement, load balancing, reinforcement learning**

I. Introduction

Network Function Virtualization (NFV) replaces dedicated hardware middleboxes with virtualized network functions (NFs) running on commodity servers, enabling elastic scaling and fault tolerance that fixed hardware cannot provide [1]. A stateful NF's correctness depends on preserving per-flow and global state across failures and load-driven scaling events. The dominant approach to fault tolerance for stateful NFs is *replication*: a primary NF instance forwards its state updates to one or more backup (secondary) instances, which take over processing if the primary fails [2]–[4]. DEFT [5] extends this line of work with a distributed, controller-free two-phase-commit (2PC) protocol that achieves both elastic scaling and fault tolerance with strong consistency on global state updates, without the single point of failure a central controller introduces.

A. The Gap: Selection of Backup Nodes

Across this entire line of work, backup assignment is never considered as a decision with consequences. REINFORCE [4] designates a predecessor node and a backup node per NF without describing a selection rule. OpenNF [6], Split/Merge [7], StatelessNF [8], and S6 [9] all describe how state is migrated or shared once a target instance is known, never how that target is chosen in the first place. In DEFT's own reference implementation, this manifests concretely: the shipped static mapping for its 9-host evaluation topology concentrates 3 of 5 primaries onto a single backup host, with no load or health awareness whatsoever [5].

This is a consequential gap. The entire point of replication for fault tolerance is that the backup must be available and capable of taking over. If the backup itself is overloaded, unhealthy, or already serving many other primaries, the guarantees of the replication mechanism are only as good as an arbitrary assignment allows. Elastic scaling compounds this: as NF instances are added and removed under varying load, a backup pool's composition changes continuously, and a static or naively-assigned mapping cannot track it. We deliberately scope this work to the fault-tolerance half of state management, namely replication, because that is where the backup-selection decision lives. Choosing a target for state migration during elastic scaling is a related but distinct decision; our assignment formulation could extend to it, but we do not evaluate it here.

B. Contributions

We present iDEFT (intelligent DEFT), addressing backup-node selection directly and using DEFT as the concrete system under study though the problem and our methods generalize to any primary/backup replication scheme. Specifically, we:

1) Formulate backup selection as a feasibility and load-aware assignment problem, and design three methods: an exact globally-optimal assignment via the Hungarian algorithm under capacity constraints, a deterministic multi-criteria greedy heuristic, and a learned PPO policy with invalid-action masking during training.
2) Build a reproducible evaluation harness that drives DEFT's real, unmodified 2PC backup implementation under controlled concurrent load via XML-RPC (§IV), and evaluate every method against DEFT's shipped static mapping, naive baselines (random, round-robin and a fairer randomized-offset variant), and decentralized hash-based placement (rendezvous hashing and weighted consistent hashing, the NFV analog of CRUSH/Dynamo), across four topology scales, multi-hub topologies, and four metrics (tail latency, throughput, failure rate, selection overhead), with rigorous multi-seed statistics.
3) Show that feature-aware selection reliably avoids degraded hosts and reduces tail latency once hosts differ in quality, that the simple deterministic heuristic tracks the Hungarian-optimal bound at every scale, and, through a systematic sensitivity analysis over the heuristic's scoring weights and load-decay constants (§V-G), that these constants are principled, near-optimal choices rather than arbitrary ones.
4) Report the measured behavior of RL on this problem: PPO matches the feature-aware methods up to 32 hosts and, at 128 hosts, avoids degraded hosts completely while load-balancing less tightly than the heuristic. A cross-algorithm check with DQN (§V-F) reproduces the same concentration behavior at scale under the same remedy, which points at the problem formulation rather than at a single algorithm.

II. Related Work

A. Elastic and Fault-Tolerant State Management

NF state management for elastic scaling follows two strategies. State migration moves NF state between instances. Split/Merge [7] splits state into per-flow and global parts; global updates merge periodically and stay eventually consistent, but in-flight traffic can be dropped during migration. OpenNF [6] offers move, copy, and share operations for different consistency levels. Both systems need a central controller to coordinate migration.

Migration avoidance instead decouples state from processing. StatelessNF [8], [10] keeps state in a remote data store, adding per-packet latency and a single point of failure. S6 [9] spreads state across a Chord-like key/object layer [11], reducing, but not eliminating, migration through caching and replication.

NF fault tolerance instead relies on replication and checkpointing. Pico Replication [2] checkpoints frequently [12] for loss-free recovery, but does not handle in-flight packets arriving during a checkpoint. FTMB [3] checkpoints infrequently and logs and replays packets instead, favoring throughput over checkpoint frequency. REINFORCE [4] combines both: packets are buffered at a predecessor node and replayed to a backup on failure, with lazy checkpointing except for non-deterministic packets [13], which force a strict checkpoint. All three assume a backup already exists for each primary; none describe how it is chosen.

DEFT [5] spans both strands: one distributed 2PC protocol handles elastic scaling and fault tolerance together, with no central controller (unlike [6], [7]). But it inherits the fault-tolerant systems' gap: backup assignment is a bookkeeping step in its Failure Detection Unit, not a studied decision.

B. Reinforcement Learning for NFV Resource Management

A separate line of work applies reinforcement learning to NFV resource management, almost entirely for primary placement, scaling, or embedding, not backup/replica selection. Cand-RL [14] uses PPO for availability-aware Service Function Chain (SFC) placement. DDQN-VNFPA [15] uses Double Deep Q-Networks for VNF placement. Two recent surveys, on RL for virtual network embedding [16] and on RL for network slice scaling and placement [17], both catalogue placement, scaling, and embedding as RL's NFV subproblems; backup/replica selection for fault tolerance appears in neither.

C. Replica or Backup Selection

Where the elastic and fault-tolerant state-management systems above specify how state is replicated or migrated, they take the choice of backup host as given, left to an implicit default rather than chosen from the candidate hosts' resource state. The few efforts that come near replica-like selection approach it from a different angle. Kibalya et al. [18] place stateful VNFs, but the state guiding that placement is a primary's own execution state, not a fault-tolerance relationship between a primary and a distinct backup. Avgeris et al. [19], the closest in spirit, reposition VNFs once a failure or contention event has already been detected, a reactive remedy applied after the fact, whereas we choose a secondary proactively, as replication is being established.

The idea that replica placement deserves algorithmic attention in its own right is, by contrast, long established outside NFV, in distributed storage. CRUSH [20] computes failure-domain and load-aware placement without a central lookup table, and Dynamo [21] spreads replicas via consistent hashing with capacity-aware preference lists; both make "which node holds the copy" a first-class, criteria-driven decision. Their underlying mechanism, decentralized weighted hashing (rendezvous/HRW and consistent hashing), carries over directly to NF backups, which is why we adopt it as a baseline family

(§V-H). We find that such stateless schemes trail a lightly-coordinated selector on load-balancing. To our knowledge, no prior work brings this first-class treatment to *proactive backup-NF selection* for stateful NFV fault tolerance, the gap this paper addresses.

III. Backup-Selection Methods

Let P be the set of primary NFs and H the set of all hosts (a superset of P; any host not currently a primary for a given flow, is a feasible backup for that primary, including hosts that are primaries for other flows). For each primary $p \in P$ we must choose a backup $b \in H \setminus \{p\}$. Each candidate host h has a dynamic resource state: health $\mathrm{health}(h) \in [0,1]$, remaining capacity $\mathrm{cap}(h)$ out of $\mathrm{cap}_{\max}(h)$, CPU and storage availability, and a topology-derived hop-count and latency to p. Concretely, per host and per random seed these are drawn independently as $\mathrm{cap}_{\max} \sim \mathcal{U}\{50,\dots,100\}$ (integer), with cap initialised to $\mathrm{cap}_{\max}$; $\mathrm{health} \sim \mathcal{U}[0.5,1.0]$; and CPU and storage availability each $\sim \mathcal{U}[0.4,1.0]$; latency is $1.5\,\mathrm{ms} \times \mathrm{hops}$. Degraded hosts in the heterogeneous scenarios (§IV) instead pin $\mathrm{health} = 0.05$, $\mathrm{cap} = 2$, $\mathrm{cap}_{\max} = 60$ (capacity ratio ≈ 0.033). Infrastructure hosts (client, stamper) are never sampled as backups.

From synthetic features to real telemetry. These four features are deliberately chosen to correspond to signals a production NFV orchestrator already exposes, so the abstraction is a stand-in for measurable metrics rather than an idealization unavailable in practice. health maps to a composite liveness/health score: failure-detector confidence, recent error/drop rate, or hardware SMART/thermal indicators, as surfaced by an ETSI NFV-MANO VNF health indication or a Kubernetes readiness/liveness probe. The capacity ratio $\mathrm{cap}/\mathrm{cap}_{\max}$ maps to headroom in the NF's dominant scaling resource: free flow-table or connection-tracking entries, session/NAT-binding slots, or admission-control headroom against a provisioned limit. CPU and storage availability map to standard hypervisor/OS utilization ($1 - \mathrm{util}$, free state-store fraction) exported by cAdvisor, node-exporter, or libvirt; hop-count and latency map to the SDN controller's topology view and active RTT probing. Validating the selection against such real workload telemetry, rather than the synthetic draws used here, remains future work (§VI).

A. Baselines

Static (DEFT's shipped mapping). The hand-assigned mapping DEFT ships with for its 9-host reference topology, regenerable on demand for direct comparison.

Concentrated. Every primary assigned to the same single feasible backup: a scale-independent, maximally naive worst case, generalizing DEFT's own static mapping's tendency to over-concentrate load onto one host.

Random. A uniformly random feasible backup per primary, seeded for reproducibility: a sanity floor any non-random method should beat.

Round-robin. Cycles through the sorted list of all feasible hosts, skipping a candidate only when it equals the primary itself. Load-balances blindly, with zero awareness of host health, capacity, or latency. We also evaluate a *randomized-offset* variant that begins the cycle at a random position per run, so its degraded-host exposure reflects the degraded fraction rather than an artifact of the fixed host ordering (§V-H).

Rendezvous hashing (HRW) and consistent hashing. *Plain HRW* assigns each primary to the feasible backup with the highest deterministic hash weight $\mathrm{hash}(p,c)$: fully decentralized (no coordinator, no cross-primary state) and load-spreading in expectation, but health/capacity-unaware, the NFV analog of a consistent-hashing replica placement. *Weighted HRW* weights the hash by $\mathrm{health}(c) \cdot \mathrm{cap}(c)/\mathrm{cap}_{\max}(c)$ (in the spirit of CRUSH/Dynamo node weights), making it quality-aware while remaining decentralized. *Weighted consistent hashing* is a second decentralized quality-weighted point: a Dynamo-style hash ring with capacity/health-proportional virtual nodes (a host whose weight rounds to zero, e.g. a degraded host, places none and is never selected). Unlike the heuristic, none of these apply an inter-primary load penalty, so they spread only statistically.

B. Optimal: Capacity-Constrained Hungarian Assignment

To obtain an upper bound the heuristic and RL policy can be judged against, we solve the assignment problem exactly, using the same multi-criteria score $\mathrm{score}(p,c)$ defined in the next subsection (Eq. (1): a weighted combination of health, remaining capacity, latency, and hop-count). Each feasible backup h is expanded into $k = \lceil |P|/|H_{\mathrm{feas}}| \rceil$ capacity-constrained columns, where H_{feas} is the set of backup-eligible hosts (all hosts except the client and stamper); the cost-matrix entry for primary p and column j (backup h) is $-\mathrm{score}(p,h)$, or a large fixed penalty (10^6) if h is infeasible for p (including $h = p$). Because backup-eligible hosts always outnumber primaries in our topologies, $k = 1$ at every scale evaluated here ($\lceil 5/7 \rceil$, $\lceil 10/16 \rceil$, $\lceil 20/32 \rceil$, $\lceil 80/128 \rceil$ for the 9/16/32/128-host topologies, all equal to 1), so the solver assigns each primary to a distinct backup and the optimum attains a uniform maximum load of one; the $k > 1$ machinery is retained for denser regimes in which primaries would outnumber feasible backups. We solve this exactly via the Hungarian algorithm [22] (`scipy.optimize.linear_sum_assignment` [23]), yielding a globally optimal assignment under the capacity constraint, something no greedy or online method can guarantee in general. Solving it requires the complete cost matrix over all primaries and candidate backups at once, which in turn requires a party with a global view to gather that state and run the solver before any assignment is made. This centralized computation is in tension with DEFT's distributed, controller-free design (§I), so we use the Hungarian assignment strictly as an upper-bound benchmark, not as a deployment candidate.

C. Heuristic: Multi-Criteria Greedy Scoring

For each primary (processed in a fixed order), the heuristic scores every feasible candidate c as

$$\begin{aligned}\mathrm{score}(p,c) = w_h \cdot \mathrm{health}(c) + w_k \cdot \frac{\mathrm{cap}(c)}{\mathrm{cap}_{\max}(c)} \\ - w_\ell \cdot \hat{\ell}(p,c) - w_d \cdot \hat{d}(p,c)\end{aligned} \tag{1}$$

where $\hat{\ell}$ and $\hat{d}$ are the min-max-normalized latency and hop-count from p to c among p's candidates, and default weights $(w_h, w_k, w_\ell, w_d) = (0.35, 0.35, 0.20, 0.10)$ favor health and capacity, which are what actually differentiate candidates once a topology has heterogeneous backup quality. The greedy assignment picks $\arg\max_c \mathrm{score}(p,c)$ for each primary in turn, then applies a load penalty to the chosen host ($\mathrm{cap} \mathrel{-}= 20$; $\mathrm{health} \mathrel{*}= 0.95$) before moving to the next primary. This penalty is the mechanism that spreads load across the backup pool without any learning component.

D. RL: PPO with Invalid-Action Masking

We train a Proximal Policy Optimization [24] agent (`stable-baselines3` [25] over a `gymnasium` [26] environment) on the same scoring objective as the heuristic. One episode assigns a backup to every primary in sequence, under one random resource realization; the observation is each host's $[\mathrm{health}, \mathrm{cap}/\mathrm{cap}_{\max}, \mathrm{cpu}, \mathrm{storage}]$ concatenated with a one-hot encoding of the currently-active primary, and the action space is $\mathrm{Discrete}(|H|)$ (observation dimension $4|H| + |H|$). The reward is $\mathrm{score}(p,c)$ for a feasible choice, minus an overload penalty of -2 for each primary already assigned to that backup (i.e. $-2(\ell-1)$ when the choice makes host c the ℓ-th primary's backup), and a large fixed penalty (-10) for an infeasible choice, so invalid actions are discouraged *during training*, not merely corrected at inference time as in a naive post-hoc-masking design. A fresh random resource realization is drawn every episode so the learned policy generalizes across resource realizations rather than memorizing one instance. At inference, the deterministic policy is rolled out once per primary, falling back to the best-scoring feasible candidate in the rare case the policy still selects an infeasible action.

Table I lists the full per-model training configuration; all other PPO settings are `stable-baselines3` defaults (learning rate 3×10^{-4}, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip range 0.2, 10 optimization epochs per update). Each model is trained for its own host count, so a differently-sized topology requires a freshly trained model (§VI); the two 128-host variants are the collapsed default-hyperparameter baseline and the reported fix of §V-E, both released as checkpoints. The 128-host *intermediate* model (which only partially improved the spread) is listed for completeness but not shipped. §V-F additionally trains a DQN agent on this same environment and reward, as a cross-algorithm check on the load-balancing behavior PPO exhibits at large host counts.

TABLE I
PPO TRAINING CONFIGURATION PER MODEL. UNLISTED SETTINGS ARE `STABLE-BASELINES3` DEFAULTS. $[a, a]$ DENOTES A TWO-HIDDEN-LAYER MLP POLICY AND VALUE NETWORK OF WIDTH a.

Model (scale)	Timesteps	ent. coef	net arch	n_{steps}	batch
9-host	50 k	0.01	[128, 128]	2048	64
16-host	80 k	0.01	[128, 128]	2048	64
32-host	100 k	0.01	[128, 128]	2048	64
128-host (collapsed)	400 k	0.00	[64, 64]	2048	64
128-host (intermediate)	600 k	0.01	[128, 128]	2048	64
128-host (final, reported)	1 M	0.02	[256, 256]	4096	256

IV. EXPERIMENTAL METHODOLOGY

A. Testbed: RPC-Isolation, Not Physical Mininet

We measure backup-selection quality by driving the *real, unmodified* `Backup` and `Primary` 2PC classes from DEFT's implementation directly over XML-RPC, with each backup host running as its own OS process on localhost. This is a deliberate departure from a physical Mininet network path, which we attempted first: the development environment's kernel lacked the `openvswitch` kernel module and the `htb` traffic-control queueing discipline, no SDN controller was included in the original repository, and, decisively, `TCLink` bandwidth throttling did not engage on the available kernel, so there was no real resource constraint for contention to manifest against over the physical network path. The RPC-isolation testbed, by directly exercising the actual backup RPC server under controlled concurrent load, gives a more direct and reproducible measurement of the contention mechanism in question, at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss); see §VI.

Each primary fires a *burst* of simultaneously in-flight RPC calls rather than one sequential call-then-wait stream, matching how concurrent primaries would load a shared backup in production. Three implementation pitfalls were identified and corrected during harness development: (1) running backup servers as threads within one process shares a GIL, confounding the very contention effect being measured, so servers now run as separate OS processes; (2) creating a fresh RPC connection per call forces a new TCP handshake every time, dominating measurement with connection-thrash noise rather than queueing/RPC latency, so each burst worker keeps one persistent connection for all its calls, with retry-with-jittered-backoff on transient connection resets; and (3) an early version built a single host-resource context per seed and passed that same object to every method, but the heuristic and RL selectors apply an in-place load-decay penalty to each host they pick, so any method evaluated after them scored against an already-depleted context and its measured mapping depended on execution order, so each method now receives its own copy of the context.

B. Scenarios

Homogeneous (9 hosts). DEFT's real reference topology, 5 primaries, synthetic resource values drawn independently per random seed with no engineered heterogeneity.

Scaled heterogeneous (16, 32, 128 hosts). Synthetic star topologies at increasing scale (10/20/80 primaries; 6/12/48 backup-eligible hosts), with $\lfloor |H \setminus P|/3 \rfloor$ of the backup-eligible hosts deliberately degraded (health $= 0.05$, capacity ratio ≈ 0.03, plus an artificial +50 ms per-call server-side delay simulating a genuinely overloaded machine). This isolates whether feature-aware methods avoid known-bad hosts, independent of load-balancing alone, and whether the finding holds as the candidate pool grows by roughly an order of magnitude.

C. Statistical Methodology

We draw multiple independent topology/resource realizations (context seeds) rather than repeating trials of one fixed instance, since repeated trials of a single instance only capture RPC-timing noise, not whether a result generalizes across problem instances; reported numbers aggregate across all (context seed $\times$ trial) combinations. Per-trial method execution order is randomized so no method is systematically favored or disadvantaged by background system load at a particular point in a long run. We use the two-sided Mann-Whitney U test for pairwise significance and rank-biserial correlation as an accompanying effect size, since with sample sizes in the tens of thousands, p-values are almost always significant regardless of practical magnitude ($|r| \approx 0.1/0.3/0.5$ conventionally denote small/medium/large effects). We report bootstrap 95% confidence intervals on the mean (5000 resamples).

V. Results

A. Homogeneous Scale

Table II reports per-call latency across 5 context seeds $\times$ 3 trials (200 calls/primary, burst=10). Consistent with the preamble above, we read these comparisons by effect size rather than by p: `round_robin` vs. `optimal` ($r = +0.031$) and `heuristic` vs. `PPO` ($r = +0.036$) are both negligible, and `round_robin` vs. `PPO` does not even reach significance ($r = +0.008$); `round_robin` vs. `heuristic` is small ($r = -0.029$). `baseline`/`concentrated` vs. everything else is small-to-medium ($r \in [0.08, 0.40]$). At this small, homogeneous scale, any method that avoids gross load concentration performs similarly well: the feature-aware methods show no meaningful advantage over blind round-robin, since (as Table IV confirms) contention itself, not feature-awareness, drives the difference here; the real gap is between naive-concentration policies and everything else.

B. Scaled Heterogeneous: 16 Hosts

Table III isolates *feature awareness* specifically: 2 of 6 backup-eligible hosts are deliberately degraded. `concentrated`, `heuristic`, `optimal`, and `PPO` *never* route a primary to a degraded host across any of 5 seeds; `round_robin` hits a degraded host on 2 of 10 assignments in *every* seed (it cycles blindly with no way to know which hosts are bad); `random` is inconsistent (0 to 4 of 10, depending on seed). `round_robin` vs. `heuristic` is small-to-medium ($r = -0.1465$); `heuristic` vs. `optimal` is negligible-to-small ($r = +0.0586$, heuristic marginally behind optimal); `heuristic` vs. `PPO` is negligible ($r = +0.0673$); `optimal` vs. `PPO` does not reach significance ($r = +0.0089$). `concentrated` vs. everything is large ($r \in [0.47, 0.56]$). This is the core empirical result: feature-aware backup selection provides a real, mechanistically explained, statistically robust improvement over blind load-balancing once hosts differ in quality, the realistic case for elastic scaling in production, where backup capacity is added and removed under varying load and health conditions, not a static uniform pool.

TABLE II
HOMOGENEOUS SCALE (9 HOSTS, 5 PRIMARIES), $n = 15{,}000$ CALLS/METHOD.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
baseline	57.33	10.67	41.24	1071.77
concentrated	94.99	6.81	221.69	3125.54
random	45.98	12.35	47.50	1063.38
round_robin	35.67	18.46	54.27	1035.42
heuristic	41.06	17.48	50.68	1054.36
optimal	37.00	19.63	56.03	1034.23
PPO	36.20	18.88	54.47	1038.38

TABLE III
SCALED HETEROGENEOUS, 16 HOSTS (10 PRIMARIES), $n = 30{,}000$ CALLS/METHOD.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	157.37	4.89	1029.37	3361.56
random	124.97	16.77	360.19	1069.84
round_robin	106.94	22.41	360.26	1040.29
heuristic	39.11	19.55	53.68	1050.06
optimal	37.25	21.46	54.44	1036.45
PPO	37.71	21.47	56.97	1039.31

C. Mechanism Check: Latency vs. Backup Load

Table IV pools latency by `backup_load` (the number of primaries sharing a given backup) across all methods and seeds, isolating whether sharing itself, independent of which method produced the sharing pattern, drives latency. Mean latency rises with load in every scenario we ran, and by load 4 in the homogeneous scenario p99 has roughly tripled relative to load 1, consistent with TCP accept-queue backlog overflow at Python's `SimpleXMLRPCServer` (default backlog, single-threaded) once enough concurrent connections pile onto one backup; this is a genuine architectural property of DEFT's unmodified backup implementation, not a benchmark artifact. The mean is the stable signal here: the load-3 p99 in particular varies by nearly $3\times$ across runs of this identical configuration, since it is a tail statistic over the smallest pool in the table. In the 16-host scenario, load 9 (`concentrated` piling 9 of its 10 primaries onto one backup, the tenth being that backup's own primary) shows sharply degraded mean and p99 relative to load 1–2, reproducing the same mechanism at a more extreme sharing ratio.

TABLE IV
LATENCY VS. BACKUP LOAD, POOLED ACROSS ALL METHODS/SEEDS.

Scenario	Load	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
Homog.	1	35.35	17.94	52.07	1036.52
	2	53.44	10.23	49.57	1074.48
	3	74.77	8.14	221.96	1090.20
	4	111.85	6.12	1018.58	3140.93
16-host	1	58.19	20.67	355.99	1038.28
	2	49.88	13.35	49.22	1071.96
	3	238.40	14.08	361.21	1390.65
	9	172.04	4.59	1036.05	3374.23

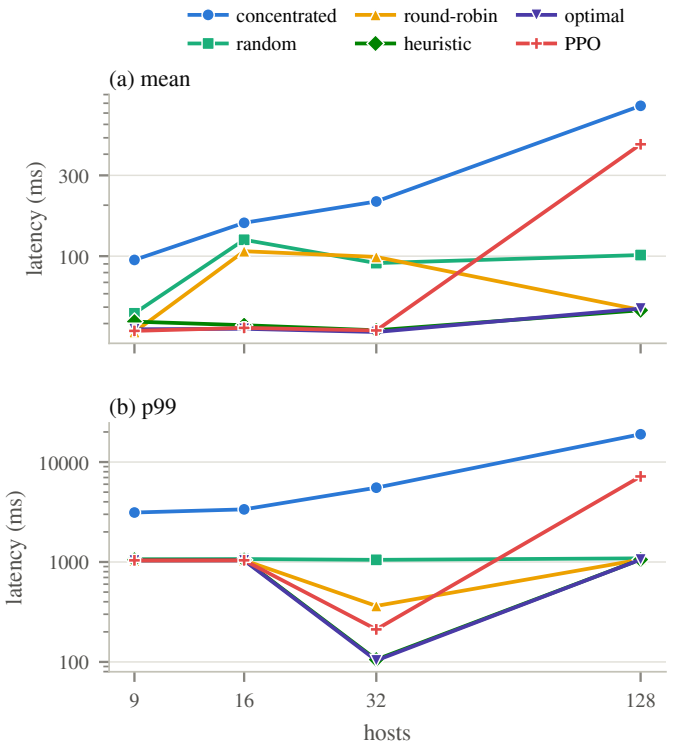


Fig. 1. Per-call latency vs. topology scale: (a) mean, (b) p99 (log axes). The 9-host point is the homogeneous scenario (its `baseline` method, which exists only there, is omitted); 16/32/128 hosts are the scaled heterogeneous scenarios of Tables III, V, and VI. `heuristic` and `optimal` stay flat across the sweep; `concentrated` degrades monotonically; `PPO` tracks the feature-aware methods until 128 hosts.

D. Scalability Trend: 32 Hosts

Figure 1 plots mean and p99 latency across the full four-point scale sweep. Table V reports the 32-host point (3 context seeds $\times$ 2 trials, 150 calls/primary, burst=8, reduced from the 16-host protocol to keep the larger run tractable on a single machine; see §VI). The 16-host finding holds at $2\times$ scale: `concentrated`, `heuristic`, `optimal`, and `PPO` again never route to a degraded host in any seed; `round_robin` hits one in 4 of 20 assignments, every seed. `round_robin` vs. `heuristic` remains small ($r = -0.1422$), similar in direction and magnitude to the 16-host result; the three feature-aware methods remain statistically close to each other ($r \in [-0.07, 0.04]$ pairwise); `concentrated` vs. everything

TABLE V
SCALED HETEROGENEOUS, 32 HOSTS (20 PRIMARIES), $n = 18{,}000$ CALLS/METHOD.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	210.35	4.28	1053.62	5538.62
random	91.00	24.49	358.65	1050.90
round_robin	98.97	28.78	359.29	363.41
heuristic	36.49	26.49	58.41	105.63
optimal	35.65	27.70	58.59	103.48
PPO	36.41	25.86	54.58	211.59

TABLE VI
SCALED HETEROGENEOUS, 128 HOSTS (80 PRIMARIES), $n = 240{,}000$ CALLS/METHOD.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	772.41	4.05	3325.13	18971.94
random	101.54	32.20	357.33	1087.47
round_robin	48.13	30.95	69.15	1058.35
heuristic	47.92	30.97	67.57	1058.17
optimal	49.06	31.38	71.57	1057.86
PPO	457.86	26.65	232.01	7178.12

is large and *stronger* than at 16 hosts ($r \in [0.71, 0.76]$, vs. $[0.47, 0.56]$), since piling 19 of the 20 primaries onto one backup pushes the single-threaded backup server further into queue backlog. Notably, `round_robin`'s hit rate is not a random draw but a fixed consequence of how Python's alphabetical host-name sort aligns with the degraded hosts; §V-E shows this coincidence flips at 128 hosts.

E. Scalability Trend Continued: 128 Hosts

Table VI reports the 128-host point at the original 16-host trial count (5 context seeds $\times$ 3 trials, 200 calls/primary, burst=10) rather than a reduced protocol, to keep this the most statistically comparable large-scale point. This run required a benchmark-harness hardening: the original 3-attempt retry budget assumed a worst case of roughly 10 primaries sharing one backup; at 79 primaries sharing one backup (`concentrated`), Python's default `SimpleXMLRPCServer` accept backlog (5) was overwhelmed badly enough that calls exhausted the retry budget and crashed the experiment outright. We hardened this to 8 attempts with jittered exponential backoff, recording a call that still fails after all retries as a failure (a 10 s sentinel latency) rather than crashing, the same robustness a real production 2PC client would need. This changes the harness's failure handling, not the backup server under test.

Concentrated sits at the edge of outright failure at this scale. In one of the five seeds, calls against the single backup serving 79 other primaries exhausted the 8-retry budget and were recorded as failures at the 10 s sentinel latency; the other four completed every call. That this swings across otherwise-identical seeds is part of the finding: `concentrated` is not merely bad here but *unstable*, with mean latency more than an order of magnitude above every other method and p99 at roughly 19 s of genuine queueing delay. Using Python's default

accept backlog, DEFT's shipped backup cannot reliably serve roughly 79 concurrent primaries even with client-side retry-with-backoff, a genuine scalability ceiling of the implementation rather than a benchmark artifact.

Round-robin, heuristic, and optimal are practically indistinguishable at this scale. `round_robin`'s mapping happens to hit zero degraded-host assignments at 128 hosts (the alphabetical-sort coincidence noted in §V-D), so it is not penalized here for the mechanism this scenario is designed to test. Their means sit within about 2% of one another (47.9, 48.1, and 49.1 ms for `heuristic`, `round_robin`, and `optimal`), and since these three methods' mappings are fully determined by topology and context seed, that residual spread is RPC-timing noise rather than a quality difference. This is not a reversal of our core claim but its clearest illustration: round-robin's outcome is uncontrolled, scoring badly at 16/32 hosts and well at 128 hosts purely by naming coincidence, whereas heuristic and optimal avoid degraded hosts *by construction* regardless of scale, naming, or noise.

RL: an attempted fix, with partial success. The original `rl_model_scaled128` (400,000 timesteps, `stable-baselines3`'s default entropy coefficient of 0 and a $[64, 64]$ network) converged to a near-degenerate policy: across the five context seeds it used only 1–5 distinct backups out of 80, funnelling almost every primary onto the same single host (`w101`) and triggering the same accept-backlog failure as `concentrated`. Since PPO's entropy bonus is zero by default, nothing discourages a policy from locking onto one action early once it looks decent on average, a plausible root cause, compounded by a $[64, 64]$ network's limited capacity to represent a policy conditioned on which of 80 one-hot-encoded primaries is active. We made two escalating retraining attempts: an intermediate one (entropy coefficient 0.01, network $[128, 128]$, 600k timesteps) that improved the spread only partially, and the fix that produced the reported model (entropy coefficient 0.02, network $[256, 256]$, larger rollout and batch sizes, 1M timesteps, the `PPO` row of Table VI). Both configurations are shipped as reproducible checkpoints.

The entropy/capacity-scaled configuration produces two measured outcomes. On host quality, the policy avoids degraded hosts completely: zero degraded-host assignments across all 5 seeds (0 of 400), matching heuristic/optimal and generalizing across every seed tested. On load balancing, it spreads the 80 primaries across 24–30 distinct backups (against the heuristic's 74–76), with the tighter seeds concentrating up to 28 of the 80 on one host (peak per-backup load 9–28), versus heuristic/optimal's near-uniform 1–2. This concentration appears only in the tail: `PPO`'s median latency (26.6 ms) is the lowest of any method in Table VI, while its most-loaded host drives mean 457.86 ms and p99 7178.12 ms. The gap is thus a tail effect on a minority of the load rather than a broad regression, and it is well above the original collapse, which crashed the harness entirely.

The mechanism behind the load-balancing gap is visible in the reward: the health and capacity terms label which hosts are bad and the network learns that signal, whereas concentration is discouraged only implicitly, through the health/capacity decay, and never by a direct anti-concentration term. Host-quality discrimination and load-balancing precision are therefore acquired at different rates as the action space grows. Reward shaping that penalizes load concentration directly is a natural way to address this; we did not attempt it here, and leave a fuller study of RL for backup selection in this setting to future work (§VII).

TABLE VII
PPO VS. DQN POLICY STRUCTURE, 5 CONTEXT SEEDS PER SCALE. "DISTINCT" IS THE RANGE OF DISTINCT BACKUPS A POLICY ASSIGNS ITS PRIMARIES ACROSS SEEDS (HIGHER IS A WIDER SPREAD), "PEAK" THE LARGEST NUMBER OF PRIMARIES PLACED ON ANY ONE BACKUP (LOWER IS BETTER), AND "DEGRADED" THE ASSIGNMENTS LANDING ON A DELIBERATELY DEGRADED HOST, OUT OF 5 SEEDS $\times$ $|P|$.

Scale	Algorithm	Distinct	Peak	Degraded
9-host	PPO	5–5 of 5	1–1	0/25
	DQN	5–5 of 5	1–1	0/25
16-host	PPO	10–10 of 10	1–1	0/50
	DQN	9–10 of 10	1–2	0/50
32-host	PPO	15–19 of 20	2–2	0/100
	DQN	12–16 of 20	2–3	1/100
128-host	PPO	24–30 of 80	9–28	0/400
	DQN	**1–3 of 80**	**45–80**	**172/400**

F. Cross-Algorithm Check: DQN

The load-balancing gap of §V-E is measured on one algorithm, which leaves open whether it reflects PPO or the problem formulation. To separate these we repeat the evaluation with Deep Q-Networks (DQN) [27], a deliberately different design point: off-policy with a replay buffer rather than on-policy, value-based rather than actor-critic, and exploring by epsilon-greedy decay rather than an entropy bonus. DQN is trained and evaluated through the same environment, reward, and inference path as PPO, so only the learning algorithm differs.

We score both policies *structurally*, by degraded-host assignments, distinct backups used, and maximum per-backup load (Table VII). These are properties of the produced mapping rather than of RPC timing, which matters here for two reasons. First, load-balancing precision is itself a structural property, so this measures the quantity in question directly rather than through its latency consequence, and Table IV has already established backup load as what drives latency under contention. Second, structural metrics are hardware-independent, so unlike per-call latency they reproduce exactly on any machine.

Up to 32 hosts the two algorithms behave alike. Both spread across nearly every available backup at 9 and 16 hosts, and at 32 hosts DQN stays within a few backups of PPO at a comparable peak load, with a single incidental degraded-host assignment. Reaching that required the same class of intervention PPO needed at 128 hosts: with library-default

settings DQN collapsed to 2–4 distinct backups of 20, and widening exploration (exploration fraction $0.5 \rightarrow 0.7$, final epsilon $0.05 \rightarrow 0.15$), enlarging the replay buffer (50k $\rightarrow$ 100k), and training longer (100k $\rightarrow$ 150k steps) recovered the spread reported here.

At 128 hosts DQN concentrates far more sharply than PPO. It funnels the 80 primaries onto 1–3 distinct backups with up to 80 primaries on a single host, and routes 172 of 400 assignments onto degraded hosts. PPO, with its entropy- and capacity-scaled configuration, avoids degraded hosts entirely in the same comparison while still spreading across only 24–30 backups, against the heuristic's 74–76 at a maximum load of 2 (§V-E).

What this establishes. Two algorithms that differ in exploration mechanism, policy representation, and update rule both concentrate load as the action space grows, and both respond to the same widened-exploration remedy. The behavior is therefore better attributed to the problem formulation, in particular a reward that labels host quality explicitly but discourages concentration only implicitly, than to any single algorithm. It also sharpens the practical reading of §V-E: the deterministic heuristic reaches a wider spread than either learned policy with no training, no per-scale tuning, and no exposure to this failure mode.

G. Heuristic Weight Sensitivity and Decay Calibration

§III fixes the multi-criteria heuristic's scoring weights at $(w_h, w_k, w_\ell, w_d) = (0.35, 0.35, 0.20, 0.10)$ and its load-decay mechanics (cap $-=$ 20; health $*=$ 0.95) by intuition. To show these are principled rather than ad hoc, we run a systematic sensitivity sweep: 287 weight configurations (286 points on a 0.1-step grid over the 4-simplex, plus the exact published default, which does not fall on that grid), each evaluated *structurally* (degraded-host hit rate, maximum per-backup load, and distinct backups used) across the real 9-host DEFT topology and the synthetic 16/32/128-host topologies, 5 context-seed draws each. This sweep requires no RPC calls (it evaluates `backup_selector.select_backups` directly against each synthetic resource draw), so it is exact and reproducible rather than subject to RPC-timing noise.

Degraded-host avoidance is essentially weight-independent. All 287 configurations, at all four topologies (1,148 configuration-topology pairs total), achieved 0% degraded-host usage, including the extreme case of weighting 100% on hops alone. This is a direct consequence of how starkly the artificially-degraded hosts are constructed relative to normal ones (health 0.05 vs. 0.5–1.0; capacity fraction ≈ 0.033 vs. ≈ 0.4–1.0): the gap is large enough that even a weight vector with zero net discriminative power still favors a non-degraded candidate on tie-break. The heuristic's degraded-host-avoidance guarantee is therefore robust to the exact weight choice, not a fragile artifact of the specific published weights.

Load-balancing quality is highly weight-sensitive, and its principal lever is the capacity decay term. Across the sweep, maximum per-backup load ranges from perfect (max load of 1 primary/backup, achieved whenever the capacity weight is nonzero and health/latency/hops are near zero) to severe (up to 79 of 80 primaries sharing one backup at 128 hosts, when capacity and health weights are both zero). The capacity term, realised as the flat -20 decrement applied to a chosen backup's remaining capacity after each assignment, is the dominant load-balancing lever: even a small nonzero capacity weight alone reaches near-optimal balance at every scale tested. The health decay ($\times 0.95$ per assignment) is a weaker lever toward the same end, and the latency and hops terms contribute essentially nothing to load-balancing in the topologies tested, since every backup-eligible host is exactly two hops from any primary via the shared hub and so normalizes to zero. This directly addresses whether the decay mechanics are arbitrary: they *are* the mechanism by which load spreads across the pool, and the sweep shows the -20 capacity decrement specifically is what drives it.

TABLE VIII
PUBLISHED DEFAULT WEIGHTS (w_h=0.35, w_k=0.35, w_ℓ=0.20, w_d=0.10) VS. THE BEST ACHIEVABLE (MAX LOAD = 1) AT EACH SCALE, 5-SEED AVERAGE.

Scale	Default max load	Best max load	Default backups used	Max possible
9-host (real)	1.40	1.00	4.60	5
16-host	1.40	1.00	9.40	10
32-host	2.00	1.00	18.00	20
128-host	2.00	1.00	74.80	80

The published default is a principled, near-optimal choice (Table VIII): at every scale it lands within one unit of the achievable max-load optimum and uses 90–94% of the maximum possible distinct backups. Sorted by (degraded-rate, max-load), the default configuration ranks 151st–187th of the 287 grid points across scales, a position that looks middling only because so many configurations tie at the max-load-of-1 ceiling (any configuration with a nonzero capacity weight tends to reach it); the default is a small, practically immaterial distance from that ceiling, not meaningfully behind it. The honest limitation this sensitivity analysis surfaces is not that the published weights or decay constants are poorly chosen, but that two of the four terms (latency, hops) carry negligible practical weight in the uniform-hop synthetic topologies used here, a consequence of the RPC-isolation testbed (§VI) rather than of the scoring design itself, which would plausibly matter more on a real bandwidth-constrained, variable-latency network.

The decay constants themselves are a characterized, well-behaved lever, not an ad hoc choice. To address the load-decay mechanics directly, we additionally sweep both constants (the capacity decrement over $\{5, 10, 20, 40, 80\}$ and the health-decay factor over $\{0.80, 0.90, 0.95, 0.99, 1.00\}$) with the scoring weights fixed, 5 seeds per topology (100 decrement$\times$decay$\times$topology cells). Two properties hold. First, *degraded-host avoidance is entirely decay-independent*: 0 of the 100 cells routed any primary to a degraded host, confirming the safety guarantee comes from the scoring, not from decay tuning. Second, load-balancing is a *smooth,*

monotone function of both constants: holding the health decay at its published 0.95, raising the capacity decrement through $5, 10, 20, 40, 80$ lowers mean maximum per-backup load through $2.60, 2.00, 2.00, 1.00, 1.00$ (identical at 32 and 128 hosts), and a stronger health decay moves it the same way. The published decrement of 20 is thus a deliberately conservative operating point, within one unit of the perfect-spread optimum, that leaves a small, bounded amount of load-balancing unclaimed rather than spreading maximally aggressively, which on a pool with finer non-degraded quality variation could trade host quality for balance. The published $(20, 0.95)$ is therefore a principled choice on a monotone response surface, not an arbitrary constant.

H. Decentralized Hash Baselines, a Fairer Round-Robin, and Topology Diversity

We broaden the baseline set along three axes the load-balancing and distributed-storage literature would expect, and re-examine the findings structurally (degraded-host hits and maximum per-backup load, 5-seed average, no RPC calls) in the heterogeneous scenarios where host quality differs (Table IX).

Decentralized hash placement (HRW and consistent hashing). We add three decentralized baselines in the style well established in distributed storage (CRUSH [20], Dynamo [21]): plain rendezvous hashing (`hrw`), health/capacity-weighted rendezvous hashing (`weighted_hrw`), and Dynamo-style weighted consistent hashing (`weighted_chash`). Plain HRW is health/capacity-unaware, and like round-robin its degraded-host exposure is uncontrolled: it routes primaries to degraded hosts at 16 and 128 hosts and misses them at 32 only by hash coincidence. Both *weighted* variants avoid degraded hosts entirely (0 across all scales), confirming that *quality-awareness, not centralization, is what buys degraded-host avoidance*. However, both are coordination-free (each primary hashes independently, with no load penalty propagated between assignments), so they spread load only statistically: mean maximum per-backup load is 3.0/3.6/2.2 (weighted HRW) and 2.8/3.0/3.2 (weighted consistent hashing) at 16/32/128 hosts, against the greedy heuristic's 1.4/2.0/2.0 and the Hungarian optimum's 1.0. A single coordinated greedy pass therefore load-balances materially tighter than *either* decentralized weighted-hash family at equal degraded-host avoidance, and the exact assignment is tighter still.

A fairer round-robin. The deterministic round-robin's degraded-host exposure depends on Python's alphabetical host-name sort, which happens to miss the degraded range at 128 hosts (§V-D). Adding a randomized-offset round-robin (`rr_random`) removes that artifact: with a random starting offset it hits degraded hosts at *every* scale (1.2/2.6/10.8 primaries at 16/32/128 hosts, tracking the degraded fraction), confirming that the deterministic variant's apparent 128-host safety was pure naming coincidence, and that any quality-blind load-spreader is exposed to degraded hosts in proportion to how many there are. This strengthens rather than weakens the core claim: the feature-aware methods avoid degraded hosts *by construction* at every scale, a guarantee no round-robin variant offers.

TABLE IX
DECENTRALIZED HASH BASELINES AND A FAIRER ROUND-ROBIN VS. THE FEATURE-AWARE METHODS, 5-SEED AVERAGE, HETEROGENEOUS STAR SCENARIOS (MULTI-HUB RESULTS ARE IDENTICAL; SEE TEXT). "DEG." = MEAN PRIMARIES ROUTED TO A DEGRADED HOST (LOWER IS BETTER); "LOAD" = MEAN MAX PER-BACKUP LOAD (LOWER IS A TIGHTER SPREAD).

	16-host		32-host		128-host	
Method	Deg.	Load	Deg.	Load	Deg.	Load
round_robin	2.0	1.0	4.0	1.0	0.0	1.0
rr_random	1.2	1.0	2.6	1.0	10.8	1.0
hrw	2.0	3.0	0.0	3.0	9.0	2.0
weighted_hrw	0.0	3.0	0.0	3.6	0.0	2.2
weighted_chash	0.0	2.8	0.0	3.0	0.0	3.2
heuristic	**0.0**	**1.4**	**0.0**	**2.0**	**0.0**	**2.0**
optimal	0.0	1.0	0.0	1.0	0.0	1.0

Topology diversity. To test whether the findings depend on the single-hub star, we repeat the comparison on multi-hub topologies (three hubs in a line, hosts distributed across them, so host-to-host hop-count and latency now vary). The structural outcome is unchanged: at 16-host/3-hub and 32-host/3-hub the heuristic still avoids degraded hosts entirely at a maximum load of 1.4 and 2.0 (identical to the single-hub stars), the weighted-hash baselines still avoid degraded hosts but spread loosely (max load ≈ 3), and the fair round-robin still hits degraded hosts (1.2 and 2.6). Because degraded-host avoidance and load-balancing are driven by host *quality* rather than graph *shape*, the core findings are robust to topology structure; a fuller sweep over irregular, multi-hub, and uneven-fan-out graphs remains future work (§VI).

I. Selection Overhead

"Elastic scaling" requires the selection algorithm itself to be fast: a method that matches the optimal bound on placement quality but takes seconds to compute is a materially worse deployment story than one that is comparably fast. Table X reports the mean wall-clock time to compute one full mapping, across all four topology sizes (30 repeats each, first call excluded as warmup), and Figure 2 plots the same data on log–log axes. Even the slowest method computes a full 128-host mapping in about 50 ms (`PPO`, 49.6 ms mean), and every other method in under 10 ms, negligible relative to any realistic failure-detection interval (typically hundreds of milliseconds to seconds in NF failure-recovery literature [28], [29]). `PPO` is consistently slowest (one PPO forward pass per primary, plus framework call overhead); its jump at 128 hosts (49.6 ms, versus 11.3 ms at 32 hosts) is the direct cost of the wider $[256, 256]$ network needed to fix the policy collapse described in §V-E, on top of the growth in primary count. `heuristic` and `optimal` grow roughly linearly/superlinearly with host count (the Hungarian algorithm is $O(n^3)$) but remain below 9 ms at 128 hosts. None of the feature-aware methods intro-

TABLE X
SELECTION OVERHEAD (MS) PER FULL MAPPING COMPUTATION.

Method	9 hosts	16 hosts	32 hosts	128 hosts
concentrated	0.005	0.013	0.055	0.731
random	0.021	0.029	0.084	0.639
round_robin	0.004	0.005	0.015	0.030
heuristic	0.055	0.161	0.646	7.260
optimal	0.097	0.208	0.913	8.943
PPO	1.81	3.76	11.29	49.64

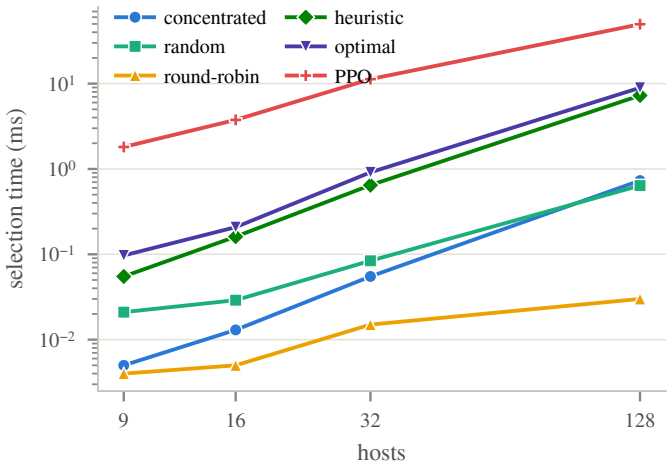


Fig. 2. Selection overhead per full mapping computation (log–log axes; data of Table X). Even the slowest method (PPO, 49.6 ms at 128 hosts) remains well below realistic failure-detection intervals.

duce a selection-latency cost that would offset their placement-quality advantage in practice.

J. Throughput and Failure Rate

Tables XI and XII complement per-call latency with the other two metrics DEFT's original evaluation reports: throughput and packet drop. We measure each primary's burst workers for a fixed 3-second wall-clock window (rather than a fixed call count) and report successful calls/second and the percentage of calls that time out or fail even after retry. This testbed is RPC-level (TCP), not DEFT's own packet-level input-buffer-overflow mechanism, which requires the Mininet/stamped-packet path this testbed deliberately bypasses; "failure rate" here is evidence of the same underlying phenomenon (an overloaded backup failing to keep up with demand), not a literal reproduction of DEFT's packet-drop metric.

At homogeneous scale we draw no conclusion from throughput. Each method contributes only 15 aggregate measurements (5 seeds $\times$ 3 trials), against the tens of thousands of individual calls behind each latency test, so the test cannot resolve the few-percent differences in Table XI: only 3 of 21 pairwise comparisons reached $p < 0.05$, none in a stable ordering, and aggregate throughput here is dominated by background system load rather than by the mapping under test. Failure rate is the one consistent signal: `concentrated`'s

TABLE XI
THROUGHPUT/FAILURE RATE, HOMOGENEOUS SCALE (5 SEEDS $\times$ 3 TRIALS).

Method	Mean throughput (calls/s)	Mean failure rate (%)
baseline	1176.0	0.38
concentrated	1158.1	0.70
random	1180.5	0.26
round_robin	1172.9	0.06
heuristic	1197.3	0.14
optimal	1159.2	0.07
PPO	1175.1	0.09

TABLE XII
THROUGHPUT/FAILURE RATE, 16-HOST SCALED SCENARIO (5 SEEDS $\times$ 3 TRIALS).

Method	Mean throughput (calls/s)	Mean failure rate (%)
concentrated	1191.4	2.57
random	1297.6	0.62
round_robin	1288.0	0.25
heuristic	1310.0	0.29
optimal	1358.8	0.16
PPO	1352.0	0.17

0.70% is the highest of any method, while the feature-aware methods stay at 0.07–0.14%.

At 16 hosts, contention is severe enough that one part of the throughput picture becomes the most robust result in this section. `concentrated` vs. everything is large and significant ($r \in [0.63, 0.78]$, all $p < 0.004$), and its failure rate (2.57%) is clearly elevated over every other method's (0.16–0.62%). Gross load concentration is therefore visible from the throughput side, not only from tail latency.

The degraded-host effect is a subtler matter, and the honest characterization is that aggregate throughput resolves it only partially. In the reported run `optimal` and `PPO` each significantly out-throughput `round_robin` ($r = 0.63$ and 0.65, $p < 0.004$), consistent with `round_robin` pairing 2 of its 10 primaries with a degraded backup in every seed; yet `heuristic` vs. `round_robin` does *not* reach significance ($p = 0.38$), despite `heuristic` avoiding degraded hosts exactly as `optimal` and `PPO` do. The mechanism explains the fragility: only 2 of `round_robin`'s 10 primaries are actually paired with a degraded backup, and their reduced individual throughput is averaged into the other 8 primaries' normal throughput at the aggregate level, leaving a signal small enough that run-to-run noise decides the verdict. Tail latency is dominated by exactly the slow calls that throughput's mean-based aggregation smooths over, and resolves the same effect consistently at every scale we tested. This is why we treat latency as the primary metric throughout and throughput as corroborating rather than replacing evidence: throughput reliably resolves gross contention, but only marginally resolves the finer degraded-host effect that this paper is actually about.

VI. Threats to Validity

RPC-isolation testbed, not the physical Mininet network (the primary external-validity gap). As described in §IV, we exercise DEFT's real backup RPC server directly rather than over a physical/emulated network path, after the available Mininet environment could not provide a genuine bandwidth constraint for contention to manifest against. This gives a direct, reproducible measurement of the contention mechanism at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss) and, in particular, DEFT's own packet-level, controller-integrated data path. A packet-level evaluation over a working OVS/SDN-controller Mininet fabric, against which the RPC-level failure rate could be cross-checked with DEFT's native packet-drop metric, is the most important remaining validation, and we treat the present study as isolating the backup-selection mechanism rather than certifying end-to-end network performance.

Single physical machine. All "hosts" are OS processes on one machine; OS-level scheduling noise is a real, visible component of tail latency. Multi-seed aggregation and large sample sizes mitigate but do not eliminate this.

Synthetic resource values and injected heterogeneity. Health, capacity, CPU, and storage availability are drawn from the configured ranges of §III, and heterogeneity is injected via a fixed $+50$ ms per-call delay and pinned degraded resource numbers, not measured from real workload telemetry; absolute magnitudes are therefore illustrative rather than calibrated. §III maps each synthetic feature to the real NFV telemetry it stands in for, but validating the selection against that live telemetry remains future work, and limits the ecological validity of the absolute (though not the relative) results.

RL configuration scales with the action space. A PPO agent's observation and action space is fixed to a given host count, so a differently-sized topology is served by its own trained model, and, as §V-E shows, the exploration and network-capacity budget must grow with the action space (the defaults that suffice at 16 and 32 hosts require scaling up at 128). We report the resulting behavior and leave reward shaping against load concentration, and a broader study of RL for this problem, to future work. For reproducibility, the reported RL numbers describe the specific released checkpoints; retraining under a different `stable-baselines3`/`PyTorch` version or host count yields a different but qualitatively similar policy. The same applies to DQN, which additionally required its own exploration widening at 32 hosts (§V-F).

Deterministic round-robin's degraded-host exposure is coincidental. Its exposure depends on how Python's alphabetical host-name sort aligns with the degraded hosts (hitting them at 16/32 hosts, missing at 128); the randomized-offset variant of §V-H confirms the exposure is real and proportional to the degraded fraction. This does not affect the load-bearing claim that heuristic and optimal avoid degraded hosts by construction at every scale, and that PPO does so empirically at every scale tested, which no round-robin variant guarantees.

Retry-policy inconsistency across scale points. The 128-host run required a harness hardening (3 to 8 retry attempts) after the original budget crashed under 80-primary contention. The 9/16/32-host results predate this fix but never exhausted the original budget in practice, so they should be unaffected, but were not re-run under the identical harness version.

Scale sweep is three points, not a continuous curve, with the 32-host point run at a reduced trial count (3 seeds $\times$ 2 trials $\times$ 150 calls vs. $5 \times 3 \times 200$ at 16 and 128 hosts) to keep it tractable. The qualitative findings hold at all three points, but a continuous sweep with fully matched trial counts would be needed for a rigorous asymptotic scalability claim.

Topology diversity is partially, not exhaustively, covered. Beyond the single-hub stars, we evaluate multi-hub topologies (§V-H), where host-to-host hop-count and latency vary, and the structural findings are unchanged, because degraded-host avoidance and load-balancing are driven by host quality rather than graph shape. Our context-seed sweep still varies only synthetic *resource* draws per seed, however; the graph shapes tested (single- and multi-hub) remain regular, and genuinely irregular graphs (uneven fan-out, heterogeneous hub connectivity, real datacenter topologies) are untested. A fuller sweep over such graphs, together with the continuous scale sweep noted above, would strengthen the generalization claim.

Two analyses are evaluated structurally rather than by measured latency. The weight and decay sweeps (§V-G) and the PPO-vs-DQN comparison (§V-F) both report properties of the produced mapping (degraded-host assignments, maximum per-backup load, distinct backups used) instead of per-call RPC latency. This keeps them exact, hardware-independent, and cheap enough to sweep 287 configurations, and it measures load-balancing precision directly rather than through its latency consequence; our mechanism check (Table IV) establishes that backup load is what drives latency under contention. The cost is that neither the ranking of weight configurations nor DQN's latency under contention has been verified end to end against measured latency alongside the other methods.

VII. Conclusion and Future Work

Every major NF state-replication system we surveyed (OpenNF, Split/Merge, StatelessNF, S6, Pico Replication, FTMB, REINFORCE, and DEFT) does not treat backup-node assignment as a decision with measurable consequences. We have closed this gap concretely with iDEFT, designing and rigorously evaluating three backup-selection strategies (an exact Hungarian-optimal bound, a multi-criteria heuristic, and a PPO policy) against DEFT's own static mapping and three naive baselines, by driving DEFT's real, unmodified 2PC implementation under controlled contention across four topology scales with statistically rigorous, multi-seed testing. Feature-aware methods reliably avoid degraded backup hosts and substantially reduce tail latency once hosts differ in quality, and a well-designed deterministic heuristic tracks the exact optimal bound closely at every scale we tested. Reinforcement learning (PPO) matches this performance up to 32 hosts and, at 128 hosts, avoids degraded hosts completely

while its load-balancing precision is lower than the heuristic's at that scale. A cross-algorithm check with DQN reproduces that concentration behavior at scale, and more sharply, under the same widened-exploration remedy, which attributes the gap to the problem formulation rather than to one algorithm. We report this measured behavior and leave a fuller study of RL for backup-node selection in this setting, including reward shaping that directly penalizes load concentration, to future work. A systematic sensitivity analysis over the heuristic's scoring weights and decay mechanics further shows that its degraded-host-avoidance guarantee holds across the entire weight space tested, that its load-balancing quality is governed almost entirely by the remaining-capacity term (whose -20 decrement is the load-spreading mechanism), and that the published default weighting is a near-optimal, non-fragile choice, so the heuristic's calibration is principled, not ad hoc.

Future work includes: reward shaping that directly penalizes load concentration for the RL policy at large scale; validating these findings over a physical or emulated network path with genuine bandwidth constraints; extending the multi-seed evaluation to vary topology structure, not just resource values; and a continuous scalability sweep with matched trial counts across an uninterrupted range of host counts.

Acknowledgment

This work builds directly on the DEFT system [5] and its publicly available implementation.

References

[1] B. Han, V. Gopalakrishnan, L. Ji, and S. Lee, "Network function virtualization: Challenges and opportunities for innovations," *IEEE communications magazine*, vol. 53, no. 2, pp. 90–97, 2015.

[2] S. Rajagopalan, D. Williams, and H. Jamjoom, "Pico replication: A high availability framework for middleboxes," in *Proceedings of the 4th annual Symposium on Cloud Computing*, 2013, pp. 1–15.

[3] J. Sherry, P. X. Gao, S. Basu, A. Panda, A. Krishnamurthy, C. Maciocco, M. Manesh, J. Martins, S. Ratnasamy, L. Rizzo *et al.*, "Rollback-recovery for middleboxes," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, 2015, pp. 227–240.

[4] S. G. Kulkarni, G. Liu, K. Ramakrishnan, M. Arumaithurai, T. Wood, and X. Fu, "Reinforce: Achieving efficient failure resiliency for network function virtualization based services," in *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies*, 2018, pp. 41–53.

[5] M. M. Shahriyar, G. Saha, B. Bhattacharjee, and R. Reaz, "Deft: distributed, elastic, and fault-tolerant state management of network functions," in *2023 19th International Conference on Network and Service Management (CNSM)*. IEEE, 2023, pp. 1–7.

[6] A. Gember-Jacobson, R. Viswanathan, C. Prakash, R. Grandl, J. Khalid, S. Das, and A. Akella, "Opennf: Enabling innovation in network function control," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 163–174, 2014.

[7] S. Rajagopalan, D. Williams, H. Jamjoom, and A. Warfield, "{Split/Merge}: System support for elastic execution in virtual middleboxes," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013, pp. 227–240.

[8] M. Kablan, A. Alsudais, E. Keller, and F. Le, "Stateless network functions: Breaking the tight coupling of state and processing," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 97–112.

[9] S. Woo, J. Sherry, S. Han, S. Moon, S. Ratnasamy, and S. Shenker, "Elastic scaling of stateful network functions," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 299–312.

[10] M. Szalay, M. Nagy, D. Géhberger, Z. Kiss, P. Mátray, F. Németh, G. Pongrácz, G. Rétvári, and L. Toka, "Industrial-scale stateless network functions," in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 2019, pp. 383–390.

[11] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: a scalable peer-to-peer lookup protocol for internet applications," *IEEE/ACM Transactions on networking*, vol. 11, no. 1, pp. 17–32, 2003.

[12] J. S. Plank, M. Beck, G. Kingsley, and K. Li, *Libckpt: Transparent checkpointing under unix*. Computer Science Department, 1994.

[13] C. Cachin, S. Schubert, and M. Vukolić, "Non-determinism in byzantine fault-tolerant replication," *arXiv preprint arXiv:1603.07351*, 2016.

[14] G. L. Santos, P. T. Endo, T. Lynn, D. Sadok, and J. Kelner, "A reinforcement learning-based approach for availability-aware service function chain placement in large-scale networks," *Future Generation Computer Systems*, vol. 136, pp. 93–109, 2022.

[15] J. Pei, P. Hong, M. Pan, J. Liu, and J. Zhou, "Optimal vnf placement via deep reinforcement learning in sdn/nfv-enabled networks," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 2, pp. 263–278, 2019.

[16] H.-K. Lim, I. Ullah, Y.-H. Han, and S.-Y. Kim, "Reinforcement learning-based virtual network embedding: A comprehensive survey," *ICT Express*, vol. 9, no. 5, pp. 983–994, 2023.

[17] N. Saha, M. Zangooei, M. Golkarifard, and R. Boutaba, "Deep reinforcement learning approaches to network slice scaling and placement: A survey," *IEEE Communications Magazine*, vol. 61, no. 2, pp. 82–87, 2023.

[18] G. Kibalya, J. Serrat, J.-L. Gorricho, D. G. Bujjingo, J. Sserugunda, and P. Zhang, "A reinforcement learning approach for placement of stateful virtualized network functions," in *2021 IFIP/IEEE International Symposium on Integrated Network Management (IM)*. IEEE, 2021, pp. 672–676.

[19] M. Avgeris, A. Leivadeas, and I. Lambadaris, "A reinforcement-learning self-healing approach for virtual network function placement," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2023, pp. 1–5.

[20] S. A. Weil, S. A. Brandt, E. L. Miller, and C. Maltzahn, "Crush: Controlled, scalable, decentralized placement of replicated data," in *Proceedings of the 2006 ACM/IEEE conference on Supercomputing*, 2006, pp. 31–es.

[21] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007, pp. 205–220.

[22] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[23] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, "Scipy 1.0: fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[25] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[26] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[28] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: measurement, analysis, and implications," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 350–361.

[29] R. Potharaju and N. Jain, "Demystifying the dark side of the middle: a field study of middlebox failures in datacenters," in *Proceedings of the 2013 conference on Internet measurement conference*, 2013, pp. 9–22.